COLLEGE ENGINEERING PROJECT · PROJECT 24 OF 55

Early Detection of Financial Fraudulent Transactions Using Graph Neural Networks

Domain: Graph Neural Networks & Cybersecurity | **Module:** project_24_financial_fraud_gnn

PROJECT ABSTRACT

The rapid growth of digital payments and online banking has increased the number of financial transactions performed every day, while also creating new opportunities for fraudulent activities. Traditional fraud detection systems often analyze transactions individually using predefined rules or conventional machine learning techniques. However, financial fraud is frequently connected to relationships between multiple accounts, devices, locations, and transactions, making it difficult to identify suspicious behavior by examining a single transaction alone. The proposed Early Detection of Financial Fraudulent Transactions Using Graph Neural Networks (GNNs) aims to address this challenge by analyzing the relationships and behavioral patterns within financial transaction networks. The system is developed using Python, machine learning, and Graph Neural Network techniques. Financial entities such as customers, bank accounts, payment methods, devices, and transactions can be represented as nodes and relationships between them as edges. This graph-based representation enables the system to study not only individual transaction characteristics but also the connections between different entities. The model can learn patterns associated with legitimate and suspicious transaction behavior and assign a fraud probability to new transactions. The major uniqueness of the proposed system is its relationship-aware fraud detection approach. Instead of treating every transaction as an independent event, the system examines connected transaction patterns to identify unusual behavior. For example, multiple accounts interacting with the same suspicious account or unusual transaction connections between different entities can provide additional evidence of potential fraud. This helps the model detect complex fraud patterns that may remain hidden when only individual transaction details are analyzed. Another important feature is early fraud prediction. The system aims to identify suspicious patterns at an earlier stage, allowing potentially fraudulent transactions to be flagged before significant financial damage occurs. A risk score can be generated for each transaction so that high-risk activities can be prioritized for further verification or investigation. The proposed system can be applied in digital banking, online payment platforms, e-commerce, and financial service applications. By combining graph-based relationship analysis with deep learning, the project provides a scalable approach for detecting evolving fraud patterns. Overall, the project demonstrates how Graph Neural Networks can transform financial fraud detection from transaction-level analysis into relationship-level intelligence, enabling earlier identification of suspicious activities and supporting more effective financial security systems.

📌 Project Metadata & Specifications

Project Title	Early Detection of Financial Fraudulent Transactions Using Graph Neural Networks
Domain / Specialization	Graph Neural Networks & Cybersecurity
Source Directory	`project_24_financial_fraud_gnn`
Backend REST API	`http://localhost:5024` (Flask Python)
Frontend Interface	Interactive Dashboard (`frontend/index.html`)
Execution Mode	Hybrid Dual-Mode (Online API & Offline Autonomous Simulation)

🏗️ System Methodology & Implementation Phases

PROJECT 24 — PHASE 1

Phase 1 — Requirements, Data and Prototype

The first phase will convert the project concept into a measurable prototype for financial/business analytics. The work begins by defining the exact input, output, user flow and evaluation criteria from the project abstract. A small requirements specification will identify the primary prediction/detection task, expected classes or target value, acceptable response time, and the form in which results must be displayed.

1. Problem Definition and Scope

Define the operational scenario, target users, system boundaries, assumptions, success criteria and non-functional requirements. Separate what the model must predict from what the application must display or recommend. Identify cases that should be rejected, flagged for review, or returned with low confidence.

2. Data Preparation

Identify a suitable dataset or approved collection procedure matching the abstract. Build a data dictionary, remove unusable records, handle missing values, normalize or standardize numerical fields where appropriate, and verify labels. For image/video/audio/text projects, define preprocessing, augmentation and quality-control rules. Split data into training, validation and test sets without leakage.

3. Baseline Model and Experiment Setup

Implement a reproducible baseline using the algorithm/model family stated in the abstract when one is specified. Establish a simple reference result before introducing additional complexity. Record dataset version, preprocessing settings, model configuration, random seed, training time and evaluation metrics so that later improvements can be compared objectively.

4. Initial Application Prototype

Build the minimum user-facing workflow needed to demonstrate the core task. The prototype should accept valid input, run preprocessing, invoke the baseline model, and return the predicted result with an understandable confidence/score where supported. Keep model logic separate from the interface so the model can be replaced without rewriting the application.

Phase 1 Deliverables

Requirements and scope note; prepared dataset or data pipeline; baseline model; experiment log; initial application/prototype; and a short Phase 1 validation report documenting what works, what remains

uncertain, and the measurements to be improved in Phase 2.

Project-Specific Phase 1 Work Items

- Create a clean feature/data dictionary directly from the variables described in the abstract.
- Establish baseline metrics before optimization and preserve the baseline for comparison.
- Document assumptions and data limitations that could affect generalization.

PROJECT 24 — PHASE 2

Phase 2 — Optimization, Integration and Validation

The second phase will turn the Phase 1 prototype into a validated project for financial/business analytics. The emphasis is on improving model reliability, connecting the complete workflow, testing realistic edge cases, and producing evidence that the final system meets the defined objectives.

1. Model Improvement

Compare relevant model configurations, tune important hyperparameters, address class imbalance where present, and evaluate whether additional features or augmentation improve generalization. Use validation data for tuning and reserve the test set for final evaluation. Prefer measurable improvements over unnecessary model complexity.

2. End-to-End Integration

Connect the data/input layer, preprocessing, inference engine, result-processing logic, database or file storage where required, and user interface. Add input validation, error handling, logging and clear status messages. If recommendations, alerts, summaries, risk scores or dashboards are part of the abstract, implement them as a separate decision-support layer around the model.

3. Testing and Evaluation

Perform functional, model, integration and usability tests. Evaluate accuracy or regression error as appropriate, along with precision, recall, F1, confusion matrix, ROC-AUC, latency or other task-specific measures where relevant. Test difficult, noisy, incomplete and out-of-distribution inputs and document failure cases instead of hiding them.

4. Demonstration and Final Evidence

Prepare representative test cases, screenshots or result tables, model-comparison evidence and a final demonstration workflow. Document limitations, data constraints, privacy/security considerations where applicable, and future extensions. The final report should distinguish measured results from proposed future work.

Phase 2 Deliverables

Optimized model; integrated application; test and evaluation report; representative evidence; limitations and risk register; deployment/demo instructions; and final project documentation suitable for academic review or demonstration.

Project-Specific Phase 2 Validation

- Evaluate false positives and false negatives separately because both affect operational usefulness.
- Test previously unseen or abnormal patterns and document how the system behaves when confidence is low.
- Include basic access control, logging and protection of sensitive input/output data in the demonstration.

Final Expected Outcome

A reproducible, tested prototype that implements the project concept described in the supplied abstract, together with measurable evaluation evidence and clear documentation of limitations, assumptions and future enhancement opportunities.

🚀 Local Deployment & Execution Guide

Follow the steps below to initialize and demonstrate the system:

1. Backend Server Setup

```
# Navigate to the backend directory cd "/Users/balamurugan/Documents/orbyte globel
/college project/project_24_financial_fraud_gnn/backend" # Install project dependencies
pip install -r requirements.txt # Run the Flask API service python3 app.py
```

The backend microservice will listen for requests at `http://localhost:5024`.

2. Frontend User Interface

- Open `/Users/balamurugan/Documents/orbyte globel /college project/project_24_financial_fraud_gnn/frontend/index.html` directly in Google Chrome, Firefox, or Safari.
- Alternatively, launch via the central **Master Launchpad Portal (`index.html`)** located at the root directory.
- The user interface is designed with **hybrid dual-mode resilience**: when the backend API is live, real-time inferences are queried; when offline, authentic client-side mathematical models simulate sensor telematics.